CBSE CLASS X • INFORMATION TECHNOLOGY (SUBJECT CODE 402)

UNIT 2: ELECTRONIC SPREADSHEET (ADVANCED) — CHAPTER 5
USING MACROS IN SPREADSHEET

Comprehensive Detailed Theory • Unabridged Practical Procedures • Code Structure & API • Solved Board PYQs & Case Studies

---

1. Concept, Definition and Advantages of Macros

Definition: In LibreOffice Calc, a **Macro** is a named, recorded, and saved sequence of commands, keystrokes, and operations that are stored for later reuse. When executed, a macro replays the exact sequence of actions automatically to perform a task.

Core Advantages of Using Macros:

- **Task Automation:** Automates repetitive keystrokes, formatting, data entry, and computation across different worksheets without manual re-entry.
- **Error Reduction:** Substantially minimizes the risk of human errors and typographical slips that inherently increase with manual repetitive tasks.
- **Time Efficiency:** Dramatically accelerates repetitive workflow cycles (e.g., formatting 48 class result sheets identically in a single click).
- **Simplification of Complex Computations:** Consolidates intricate mathematical or structural operations under a single callable name or customized function.

Crucial Operational Nuances & Limitations:

- **Mouse Movements are NOT Recorded:** The Macro Recorder logs actions taken via keystrokes, menu selections, dialog boxes, and direct cell interactions, but it **does not record mouse movements** across the window.
- **Disabled by Default:** For security reasons (to prevent malicious scripts from executing automatically), the Macro Recording feature in LibreOffice Calc is turned off by default and must be explicitly enabled.
- **Macro Security Warning:** Running macros from unknown or untrusted external sources can pose serious security hazards (such as injecting malicious code or corrupting spreadsheet data).

2. Enabling Macro Recording in LibreOffice Calc

Because macro recording is disabled by default in fresh installations of LibreOffice, users must activate it through the system options before attempting to record.

Step 1: Navigate to the Menu bar and click **Tools** → **Options...** (or press the keyboard shortcut **Alt + F12**).

Step 2: In the Options dialog box, expand the **LibreOffice** category in the left pane and select the **Advanced** sub-entry.

Step 3: On the right pane under the **Optional Features** section, locate and check the checkbox labelled **'Enable macro recording (may be limited)'**.

Step 4: Click **Apply**, and then click **OK**. The macro recording engine is now activated and ready for use in Calc.

3. Recording and Running Macros (Unabridged Procedures)

A. Unabridged Step-by-Step Procedure to Record a Macro:

Step 1: Open the spreadsheet containing the data table or result sheet where the repetitive operation is to be performed.

Step 2: Select the cell range to be formatted or manipulated (e.g., select range *B1:G5* containing header rows).

Step 3: Navigate to the Menu bar and choose **Tools** → **Macros** → **Record Macro**.

Step 4: Calc activates recording mode and displays a small floating **Record Macro** toolbar on the screen containing a single button: **Stop Recording**.

Step 5: Perform the desired actions accurately: apply formatting (e.g., set font size to 12, make text bold, change alignment, or enter standard formulas). All valid actions are logged into Calc's internal code buffer.

Step 6: When all intended actions are completed, click the **Stop Recording** button on the floating toolbar.

Step 7: The **LibreOffice Basic Macros** dialog box appears on the screen.

Step 8: In the *Save Macro In* left pane, expand **My Macros** (click the **+** icon), expand the **Standard** library, and select **Module1** (the default container for user-created macros).

Step 9: In the **Macro Name** box at the top, replace the default name (*Main*) by typing a meaningful, legal name (e.g., *FormatHeaders* or *ResultFormat*).

Step 10: Click **Save**. Calc saves the recorded sequence into the chosen module.

A.S. NOTES IT 402

B. Rules for Naming a Macro (CBSE Board Exam Must-Know):

When naming a macro in LibreOffice Calc, the identifier must strictly conform to Basic naming conventions:

- **1. Must Begin with an Alphabet:** The first character must always be a letter (e.g., A-Z or a-z). It cannot begin with a number or symbol.
- **2. No Blank Spaces Allowed:** Macro names cannot contain spaces (e.g., Format Table is invalid; use FormatTable or Format_Table).
- **3. No Special Characters Except Underscore:** Special characters, arithmetic operators, and punctuation marks (e.g., *, @, #, -, $, %, ?) are strictly prohibited. The **underscore (_)** is the only allowed symbol.
- **4. Legal Examples:** myMacro, Sort_Marks, CalculateTotal1, FormatHeading.
- **5. Illegal Examples:** solution*word (contains asterisk), my macro (contains space), 2ndMacro (begins with digit), test@calc (contains special character).

C. Unabridged Step-by-Step Procedure to Run an Existing Macro:

Step 1: Open the worksheet and select the target cell range where you want to execute the saved macro.

Step 2: Navigate to the Menu bar and choose **Tools** → **Macros** → **Run Macro...**.

Step 3: The **Macro Selector** dialog box opens on the screen.

Step 4: In the **Library** list on the left, click the **+** sign next to **My Macros**, expand **Standard**, and click on **Module1**.

Step 5: In the **Macro Name** box on the right, highlight the required macro name (e.g., *FormatHeaders*).

Step 6: Click the **Run** button. Calc instantly applies all saved actions to the active selection.

4. Internal Storage, Code Structure & the LibreOffice Basic IDE

When actions are recorded or scripted, Calc translates them into **LibreOffice Basic** programming statements. These scripts are viewed, edited, and debugged within the **LibreOffice Basic Integrated Development Environment (IDE)**.

Accessing the Basic IDE: Choose **Tools** → **Macros** → **Organize Macros** → **LibreOffice Basic...** (or click *Edit Macros*). Select the module and click **Edit** (or press **F5** to run code inside the IDE).

Subroutine vs. Function Macro Structures:

Structural Attribute	Subroutine Macro (Sub ... End Sub)	Function Macro (Function ... End Function)
Enclosing Keywords	Begins with Sub <name> and terminates with End Sub.	Begins with Function <name> and terminates with End Function.
Return Value Capability	**Cannot return a value** to a spreadsheet cell formula.	**Returns a computed value** directly to the calling formula cell.
Execution Mechanism	Run via menu (Tools → Macros → Run Macro), assigned buttons, or shortcut keys.	Called directly inside a cell formula like a regular built-in function (e.g., =Addres()).
Result Assignment	Actions are executed imperatively on the document/cells.	The final computed result **must be assigned to the function name**.

Language Characteristics & Commenting:

- **Case Insensitivity:** LibreOffice Basic is case-insensitive; keywords, variables, and function names can be entered in uppercase, lowercase, or mixed case.
- **Comments Syntax:** Explanatory documentation inside macro code begins with the keyword Rem (Remark) or a single quotation mark ('). Comments are ignored by the compiler.

5. Writing Macros that Behave as Functions & Passing Arguments

A macro can be created as a custom user-defined function to perform recurring calculations where no built-in Calc function exists. To act like a regular function, it must be declared between Function and End Function statements, and the return value must be assigned to the function's own identifier.

Example 1: Parameterless Macro Function (Displaying Standard Text):

```
Function Addres()
  Addres = "123, ABC Road, T Block, X Nagar, Delhi, India"
End Function
```

Usage in Calc: Enter =Addres() in any blank cell; Calc populates the address string.

Example 2: Macro Function Accepting Arguments (Dynamic Calculations):

The values supplied inside parentheses when calling a function are called its **Arguments**:

DAV NOTES IT-402

```
Function ValueDoubled(Optional val)
  ValueDoubled = val * 2
End Function
```

Usage in Calc: Entering =ValueDoubled(25) yields 50. Entering =ValueDoubled(C16) reads the content of cell C16 (say, 17) and returns 34.

Critical Rule — Arguments are Passed as Values:

In LibreOffice Calc, **arguments passed to a macro from a cell formula are always passed as VALUES**, not as cell references. The macro receives only the raw data value (e.g., number 17) and has no internal mechanism to determine which physical cell coordinates (e.g., *C16*) provided that value.

6. Accessing and Manipulating Cell Values Directly in Basic Code

To manipulate spreadsheet cells directly through programmed scripts rather than GUI recording, developers use the LibreOffice Universal Network Objects (UNO) object model:

1. Variable Declaration: Use the Dim statement to declare object variables:
Dim myDoc As Object, mySheet As Object, myCell As Object

2. Accessing Document: Bind the document variable to the active workbook using ThisComponent:
myDoc = ThisComponent

3. Accessing Active Sheet: Reference the active worksheet via controller or zero-based index:
mySheet = myDoc.CurrentController.getActiveSheet() *(or myDoc.Sheets(0))*

4. Accessing Specific Cell: Fetch the desired cell object by its coordinate label string:
myCell = mySheet.getCellRangeByName("C5")

5. Reading Cell Values: Use the .getValue() method to read numeric data:
val = myCell.getValue()

6. Writing Cell Values: Assign data using .setValue(<number>) or .Value = <number>:
myCell.setValue(75000)

Worked Code Example: Calculating 10% Discount from Cell B4 into Cell C4:

```
Sub CalculateDiscount()
  Dim myDoc As Object, mySheet As Object, cellAmt As Object, cellDisc As Object
  myDoc = ThisComponent
  mySheet = myDoc.CurrentController.getActiveSheet()
  cellAmt = mySheet.getCellRangeByName("B4")
  cellDisc = mySheet.getCellRangeByName("C4")
  cellDisc.Value = cellAmt.getValue() * 0.10
End Sub
```

7. Sorting Columns Using a Macro & Critical Constraints

Sorting operations can be recorded into a macro and executed across different worksheets. However, there is a fundamental constraint:

- **Identical Range Mandate:** A recorded sorting macro functions successfully **ONLY if the target data resides in the EXACT identical cell range** (e.g., *B38:C46*) across sheets.
- **Why it Fails on Different Ranges:** When a sorting operation is recorded, Calc hardcodes the specific coordinates of the selected range into the internal Basic dispatch commands. If a user attempts to run this macro on a different range (e.g., *D10:E25*), the macro will either sort the original hardcoded coordinates or fail.
- **Procedure:** Start recording (Tools → Macros → Record Macro), select the target range, choose Data → Sort..., specify Sort Key and Ascending/Descending order, click OK, stop recording, and save in Module1 as Sort_Marks.

8. CBSE Board Examination Question Bank & Model Solutions

Part A: Objective & Multiple Choice Questions (1 Mark Each)

Q1. In Calc, Arguments passed to a macro from Calc are always _____ [CBSE SQP 2022-23]
(a) Cell Reference (b) Value (c) Both a and b (d) Sheet Reference

Ans: (b) Value (Calc passes only the raw evaluated data values; the macro cannot determine which cell provided the value).

Q2. Macros are especially useful to _____ the same way over and over again. [CBSE Board 2023]
(a) repeat a task (b) reject a task (c) report a task (d) comment a task

Ans: (a) repeat a task

DAV GVM IT-402 NOTES

Q3. The code of macro begins with _____ followed by the name of the macro and ends with _____ [CBSE SQP 2024-25]
(a) Function, End (b) Sub, End Sub (c) Sub, End Function (d) Sup, End Sup

Ans: (b) Sub, End Sub (or Function, End Function for custom functions).

Q4. _____ is a text editor that allows you to create and edit macros. [CBSE SQP 2024-25]
(a) IDE (b) Notepad (c) Open Office Writer (d) MS Word

Ans: (a) IDE (Integrated Development Environment / LibreOffice Basic IDE).

Q5. Macro functions can be written to behave as regular functions by writing a/an _____ [CBSE Board 2024]
(a) Void (b) Return (c) Add-In (d) Dim

Ans: (c) Add-In (Macro functions can be compiled as Add-Ins to appear and behave as regular built-in Calc functions).

Q6. Assertion (A): In LibreOffice Calc, a Macro can be created as a function to perform repetitive calculations for which no predefined function exists.
Reason (R): A function macro is written between Function and End Function statements, can accept arguments, perform operations, and return results. [CBSE SQP 2025-26 / 2026-27]
(a) Both A and R are true, and R is the correct explanation of A
(b) Both A and R are true, but R is NOT the correct explanation of A
(c) A is true, but R is false (d) A is false, but R is true

Ans: (a) Both A and R are true, and R is the correct explanation of A

Q7. Which of the following is the correct way to record a macro in LibreOffice Calc? [CBSE Board Paper 1 (Set 4)]
(a) Tools → Macros → Record Macro (b) File → Options → Record Macro
(c) Insert → Macro → Record (d) View → Macros → Record

Ans: (a) Tools → Macros → Record Macro

Part B: Short Answer Questions (2 Marks Each)

Q8. Mention any two operations that can be performed using Macros in a spreadsheet. [CBSE SQP 2022-23, Q.12]

Ans:
1. Automatically applying uniform formatting (font, borders, shading) across multiple tabular sheets.
2. Sorting data columns in pre-set identical tabular structures or computing recurring complex financial equations.

Q9. Briefly discuss the significance of using a Macro as a function in LibreOffice Calc. [CBSE Board Paper 2 (Set 4), Q.15]

Ans: Using a macro as a function allows users to define custom mathematical or text-processing formulas that are not natively available in Calc's built-in function library. Once written between Function ... End Function, it accepts arguments, returns computed results dynamically, and recalculates automatically when cell inputs change.

Q10. What is a Macro? List any two real life situations where they can be used. [CBSE SQP 2025-26 / 2026-27, Q.15]

Ans:
• **Definition:** A macro is a saved, named sequence of commands, keystrokes, and operations used to automate repetitive spreadsheet tasks.
• **Real-Life Situations:**
1. *School Examination Results:* Applying identical table headers, borders, and grading formulas across multiple section sheets.
2. *Monthly Business Billing:* Generating standardized sales tax and discount calculations across numerous daily client invoices.

Part C: Descriptive, Long Answer & Case-Study Questions (4 Marks Each)

Q11. Case Study: A Class X student is participating in a school competition where they need to analyse the performance of different sports teams based on match scores and statistics. The student has collected data over several matches and needs to calculate averages, determine rankings, and generate charts for a presentation. [CBSE SQP 2024-25, Q.18]
a) How are Macros in Calc helpful to him?
b) How to enable Macro in Calc?
c) Is solution*word a valid macro name? Justify.
d) Mention any one operation which cannot be recorded by macro.

Ans:
a) Macros help him automate repetitive formatting, formula insertions, and calculations across multiple match sheets, saving considerable time and eliminating human errors.
b) To enable macro recording: Go to **Tools** → **Options** → **LibreOffice** → **Advanced**. Under *Optional Features*, check **'Enable macro recording (may be limited)'**, then click **Apply** and **OK**.
c) No, solution*word is **not a valid** macro name. *Justification:* Macro names cannot contain special characters or arithmetic operators like the asterisk (*). Only alphanumeric characters and underscores are permitted.
d) Mouse movements (or opening/closing windows outside the document space) cannot be recorded by the Macro Recorder in Calc.

P.S. NOTES IT-402